

Practical week-4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed. PID = %d\n", i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    // Using wait()
    printf("\nUsing wait():\n");
```

```

        printf("Child %d completed. PID = %d\n", i + 1, getpid());
        exit(10 + i);
    }
}

printf("All 3 child processes have been created.\n");

// Using wait()
printf("\nUsing wait():\n");

pid_t wpid = wait(&status);

if (wpid > 0) {
    if (WIFEXITED(status)) {
        printf("wait() detected child PID %d finished with exit status %d\n",
            wpid, WEXITSTATUS(status));
    }
}

// Using waitpid()
printf("\nUsing waitpid():\n");

for (int i = 0; i < 3; i++) {
    wpid = waitpid(pid[i], &status, 0);

    if (wpid > 0) {
        if (WIFEXITED(status)) {
            printf("waitpid() detected child PID %d finished with exit status %d\n",
                wpid, WEXITSTATUS(status));
        }
    }
}

```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

    if (wpid > 0) {
        if (WIFEXITED(status)) {
            printf("waitpid() detected child PID %d finished with exit status %d\n",
                wpid, WEXITSTATUS(status));
        }
    }
}

// Using waitpid()
printf("\nUsing waitpid():\n");

for (int i = 0; i < 3; i++) {
    wpid = waitpid(pid[i], &status, 0);

    if (wpid > 0) {
        if (WIFEXITED(status)) {
            printf("waitpid() detected child PID %d finished with exit status %d\n",
                wpid, WEXITSTATUS(status));
        }
    }
}

printf("\nParent process completed.\n");

return 0;
}

```

```

/home/laasya
laasya@laasya:~$ ls
command_executor  ossp          practical_week2  week2  week4
command_executor.c practical  skill_week4      week3
laasya@laasya:~$ nano wwek4p_task1.c
laasya@laasya:~$ gcc wwek4p_task1.c -o wwek4p_task1
laasya@laasya:~$ ls
command_executor  practical      week2  wwek4p_task1
command_executor.c practical_week2 week3   wwek4p_task1.c
ossip             skill_week4    week4
laasya@laasya:~$ ./week4_task1
-bash: ./week4_task1: No such file or directory
laasya@laasya:~$ |

```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent sleeping...\n");
        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}

```

```

laasya@laasya:~$ nano wwek4p_task2.c
laasya@laasya:~$ gcc wwek4p_task2.c -o wwek4p_task2
laasya@laasya:~$ ./wwek4p_task2
Parent process: PID = 524
Child PID = 525
Parent sleeping...
Child process: PID = 525
Child process terminating...
Parent terminating...
laasya@laasya:~$ ps -el

```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	80	0	-	5396	-	?	00:00:00	system
5	S	0	2	1	0	80	0	-	795	-	hvc0	00:00:00	init-s
0	S	0	7	2	0	80	0	-	799	-	hvc0	00:00:00	init
4	S	0	53	1	0	79	-1	-	12592	-	?	00:00:00	system
4	S	0	101	1	0	80	0	-	6252	-	?	00:00:00	system
4	S	991	118	1	0	80	0	-	5336	-	?	00:00:00	system
4	S	996	119	1	0	80	0	-	22759	-	?	00:00:00	system
4	S	0	164	1	0	80	0	-	1061	-	?	00:00:00	cron
4	S	101	165	1	0	80	0	-	2399	-	?	00:00:00	dbus-d
4	S	0	172	1	0	80	0	-	4495	-	?	00:00:00	system
4	S	0	174	1	0	80	0	-	439157	-	?	00:00:00	wsl-pr
4	S	0	192	1	0	80	0	-	781	-	tty1	00:00:00	agetty
4	S	102	193	1	0	80	0	-	55629	-	?	00:00:00	rsyslo
4	S	0	200	1	0	80	0	-	26759	-	?	00:00:00	unatte
5	S	0	293	2	0	80	0	-	797	-	?	00:00:00	Sessio
5	S	0	294	293	0	80	0	-	801	-	?	00:00:00	RelayC
4	S	1000	295	294	0	80	0	-	1520	do_wai	pts/0	00:00:00	bash
4	S	0	296	2	0	80	0	-	1675	-	pts/1	00:00:00	login
4	S	1000	338	1	0	80	0	-	5032	-	?	00:00:00	system
5	S	1000	339	338	0	80	0	-	5291	-	?	00:00:00	(sd-pa
4	S	1000	367	296	0	80	0	-	1520	poll_s	pts/1	00:00:00	bash
5	S	0	530	101	0	80	0	-	6253	-	?	00:00:00	(udev-
5	S	0	531	101	0	80	0	-	6253	-	?	00:00:00	(udev-
0	R	1000	532	295	50	80	0	-	2080	-	pts/0	00:00:00	ps

```

laasya@laasya:~$

```

```

laasya@laasya:~$ nano week4p_task2.c
laasya@laasya:~$ gcc week4p_task2.c -o week4p_task2
laasya@laasya:~$ ./week4p_task2
Parent process: PID = 546
Child PID = 547
Child process: PID = 547
Child process terminating...
Parent collected child process.
Parent terminating...
laasya@laasya:~$ ps -el

```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	80	0	-	5396	-	?	00:00:00	system
5	S	0	2	1	0	80	0	-	795	-	hvc0	00:00:00	init-s
0	S	0	7	2	0	80	0	-	799	-	hvc0	00:00:00	init
4	S	0	53	1	0	79	-1	-	12592	-	?	00:00:00	system
4	S	0	101	1	0	80	0	-	6252	-	?	00:00:00	system
4	S	991	118	1	0	80	0	-	5336	-	?	00:00:00	system
4	S	996	119	1	0	80	0	-	22759	-	?	00:00:00	system
4	S	0	164	1	0	80	0	-	1061	-	?	00:00:00	cron
4	S	101	165	1	0	80	0	-	2399	-	?	00:00:00	dbus-d
4	S	0	172	1	0	80	0	-	4495	-	?	00:00:00	system
4	S	0	192	1	0	80	0	-	781	-	tty1	00:00:00	agetty
4	S	102	193	1	0	80	0	-	55629	-	?	00:00:00	rsyslo
4	S	0	200	1	0	80	0	-	26759	-	?	00:00:00	unatte
5	S	0	293	2	0	80	0	-	797	-	?	00:00:00	Sessio
5	S	0	294	293	0	80	0	-	801	-	?	00:00:00	RelayC
4	S	1000	295	294	0	80	0	-	1520	do_wai	pts/0	00:00:00	bash
4	S	0	296	2	0	80	0	-	1675	-	pts/1	00:00:00	login
4	S	1000	338	1	0	80	0	-	5032	-	?	00:00:00	system
5	S	1000	339	338	0	80	0	-	5291	-	?	00:00:00	(sd-pa
4	S	1000	367	296	0	80	0	-	1520	poll_s	pts/1	00:00:00	bash
0	R	1000	548	295	0	80	0	-	2080	-	pts/0	00:00:00	ps

```

laasya@laasya:~$ |

```